

CREStE-Nano: Towards Mapless Visual Navigation on \$500 Hardware

A Monocular Adaptation of Learned-Reward MPPI Planning

Nishan Mani

Independent Research, UT Austin Robotics Reading Group
sambandham.dhana.lakshmi@gmail.com

Project Repository

<https://github.com/Nish989/creste-nano>

Abstract—We present CREStE-Nano, a re-implementation of the mapless visual navigation paradigm introduced by CREStE (RSS 2025) [1] on ultra-low-cost hardware: a Jetson Orin Nano, a monocular RGB webcam, and an off-the-shelf 1/10-scale RC car, with a total bill of materials of approximately \$500—roughly 20× cheaper than the original Clearpath Jackal platform. The contribution is a systems study: we replace LiDAR with monocular RGB plus Depth Anything V2 [2], replace the contrastive trajectory reward with a per-cell traversability U-Net trained under weighted binary cross-entropy on GPS-supervised corridor masks, and demonstrate that the full perception + planning loop fits within the 8 GB, 20 W envelope of the Orin Nano while running at ~5 FPS. We train on 8,331 frames of human teleoperation collected from sidewalks at Sauls Ranch, Round Rock, TX, and reproduce a corridor-vs-background margin of +0.992 across the full dataset from the released checkpoint. We further contribute a visual diagnostic that overlays the trained reward signal alongside the planner’s committed trajectory, making the gap between perception and exploitation legible. Closed-loop NIR evaluation in outdoor conditions was rained out during the planned test window and remains the headline limitation we discuss in Section VII.

I. INTRODUCTION

Mapless outdoor navigation that relies neither on HD prior maps nor on expensive LiDAR has been a long-standing target for low-cost mobile robotics. CREStE [1] recently showed that a learned reward over a BEV semantic feature volume, combined with Model-Predictive Path Integral (MPPI) [3] control, can autonomously cover a 2 km loop with a Normalized Intervention Rate (NIR) of 0.05 per 100 m. That system, however, was demonstrated on a Clearpath Jackal with a Velodyne VLP-16 LiDAR and a Jetson AGX Xavier, a stack with a list price of roughly \$10,000.

The natural question this work addresses is:

Does the CREStE paradigm survive a 20× cost reduction when the LiDAR is replaced by a single \$50 RGB webcam and the AGX Xavier is replaced by an 8 GB Orin Nano?

This question has, to our knowledge, not been systematically answered. The trade-offs are non-obvious: monocular depth is noisier than LiDAR by an order of magnitude, and the smaller Orin must amortize DINOv2 inference, depth estimation, BEV projection, the reward U-Net, and a thousand-sample MPPI rollout into a single sub-200 ms budget.

Our contribution is a complete engineering instantiation of the CREStE paradigm on \$500 of hardware, accompanied by an empirical study of the trained reward model on the full training set and a visual diagnostic that makes the perception vs. planning trade-off legible.

Contributions.

- A monocular DINOv2 [4] + Depth Anything V2 BEV projection pipeline running at ~5 FPS on a Jetson Orin Nano.
- A per-cell TraversabilityUNet trained with weighted binary cross-entropy on GPS-supervised dilated-corridor masks (Section IV-D), achieving a corridor-vs-background margin of +0.992 across 8,331 training frames (Section V).
- An end-to-end ROS 2 implementation of perception, MPPI planning, safety watchdogging, and ESP8266 PWM control on ~\$500 of commodity hardware (Section III).
- An honest two-path visualisation that overlays the trained reward signal alongside the MPPI committed trajectory, isolating the contribution of the perception from that of the off-the-shelf planner (Section VI).

II. RELATED WORK

CREStE [1] introduced the counterfactual-demonstration-trained reward model and the BEV MPPI planner that anchor this work. The original system uses LiDAR, trains the reward via InfoNCE [5] on trajectory features, and demonstrates 2 km closed-loop autonomy. We use the same overall pipeline template but substitute monocular RGB plus learned depth for LiDAR and substitute a per-cell U-Net with weighted-BCE supervision for the contrastive trajectory head.

DINOv2 [4] provides self-supervised vision-transformer features that have been shown to encode general-purpose semantic information without task-specific fine-tuning, making them a strong basis for our BEV feature volume.

Depth Anything V2 [2] provides monocular dense depth with substantially better outdoor generalisation than its predecessor. We use it for BEV projection in place of stereo or active depth.

Model-Predictive Path Integral (MPPI) control [3] is the sampling-based controller used in the original CREStE work

and retained here. We discuss its limitations in our setting (a short 1.8 m lookahead and no goal point in the offline demo) in Section VIII.

III. SYSTEM OVERVIEW

A. Hardware

The bill of materials is summarised in Table I. The total component cost is approximately \$500. All compute is on-board the Jetson Orin Nano; the ESP8266 acts solely as a serial-to-PWM bridge.

TABLE I
BILL OF MATERIALS (APPROXIMATE RETAIL USD, MAY 2026)

Component	Cost
Jetson Orin Nano Super (8 GB)	\$250
Arma Typhon Mega 1/8-scale RC chassis	\$150
EMEET SmartCam Nova 4K (monocular)	\$50
HGLRC M10 GPS	\$25
ESP8266 NodeMCU	\$5
Battery, power bank, wiring	\$20
Total	\$500

B. Software architecture

Figure 1 summarises the full software stack. The `camera_node` captures 1280×720 MJPEG frames at 30 Hz with manual exposure ($1/2000$ s) suitable for outdoor lighting. Frames are fed in parallel to DINOv2 ViT-S/14 (patch token features, 384-d per 14×14 patch) and Depth Anything V2 (relative dense depth, warped to metric via per-frame min/max clipping to $[0.5, 10]$ m). The `bev_projection_node` back-projects each pixel (u, v, d) into the camera-frame, applies a fixed extrinsic mount transform with pitch $\alpha = 15^\circ$ and height $h_c = 35$ cm, and bins the resulting points into a 64×64 BEV grid with 0.15 m cell size. Features at colliding cells are averaged.

The resulting feature volume $\mathbf{F}_{\text{BEV}} \in \mathbb{R}^{64 \times 64 \times 384}$ is consumed by the `reward_node` (the trained TraversabilityUNet, Section IV-D) and—through the same underlying ROS topic—by the `planner_node` (the MPPI controller, Section IV-E). Steering and throttle commands flow through a `safety_node` that enforces a 500 ms command timeout and an absolute throttle clamp $|u_{\text{thr}}| \leq 0.38$, and reach the ESC and steering servo via a 500 kbaud USB-CDC link to an ESP8266 microcontroller running a custom PWM-bridge firmware. The `teleop_node` accepts a PS5 DualSense controller for data collection and online human intervention, but is not required for autonomous operation: the `/autonomous_mode` flag can be published directly from a self-hosted phone-friendly web dashboard.

IV. METHOD

A. Semantic feature extraction (DINOv2)

An input image $I \in \mathbb{R}^{H \times W \times 3}$ is divided into $N = (H/14) \times (W/14)$ non-overlapping patches. DINOv2 ViT-S/14 maps each patch to a 384-dimensional token:

$$\mathbf{f}_i = \text{DINOv2}(p_i) \in \mathbb{R}^{384}, \quad i = 1, \dots, N. \quad (1)$$

The patch tokens form a feature map $\mathbf{F} \in \mathbb{R}^{h \times w \times 384}$ with $h = H/14$, $w = W/14$.

B. Monocular depth estimation

Depth Anything V2 predicts a dense relative-depth map $D \in \mathbb{R}^{H \times W}$ from a single RGB image. Since the model output is affine-invariant, we normalise per frame to a fixed physical range:

$$\hat{D}(u, v) = d_{\min} + \frac{D(u, v) - D_{\min}}{D_{\max} - D_{\min}}(d_{\max} - d_{\min}) \quad (2)$$

with $d_{\min} = 0.5$ m, $d_{\max} = 10$ m.

C. BEV projection

Each pixel (u, v) with depth $d = \hat{D}(u, v)$ is back-projected into 3D using calibrated camera intrinsics (f_x, f_y, c_x, c_y) :

$$X = \frac{(u - c_x)d}{f_x}, \quad Y = \frac{(v - c_y)d}{f_y}, \quad Z = d. \quad (3)$$

A fixed extrinsic mount transform (height h_c , pitch α) maps camera coordinates to robot coordinates $(x_r, y_r, z_r)^\top = R_\alpha(X, Y, Z)^\top + (0, h_c, 0)^\top$. The grid index in the BEV plane (resolution $r = 0.15$ m/cell, grid $W = H = 64$) is:

$$b_x = \left\lfloor \frac{x_r}{r} + \frac{W}{2} \right\rfloor, \quad b_z = \left\lfloor \frac{z_r}{r} \right\rfloor. \quad (4)$$

The semantic feature at each pixel is splatted into the corresponding BEV cell. When multiple pixels map to the same cell, the features are averaged.

D. Traversability learning (U-Net)

We learn a per-cell traversability map directly from human teleoperation demonstrations without manual labels. The reward model is a small spatial U-Net f_ψ over the BEV feature volume:

$$f_\psi : \mathbb{R}^{H \times W \times D} \rightarrow \mathbb{R}^{H \times W}, \quad H = W = 64, \quad D = 384. \quad (5)$$

The architecture (base width $C = 64$) is encoder–bottleneck–decoder: two $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$ blocks at widths $(D \rightarrow C)$ and $(C \rightarrow 2C)$, each followed by 2×2 max-pool; a bottleneck block at $(2C \rightarrow 4C)$; mirrored decoder with transposed 2×2 convolutions and skip connections; and a 1×1 head reducing $C \rightarrow 1$.

a) *Label construction.*: For each frame at time t with recorded steering $s_t \in [-1, 1]$ we build a forward rollout in BEV coordinates with step size $\delta_z = 1.5$ cells, lateral step $\delta_x = 1.0 \cdot s_t$ cells, and horizon $T_{\text{label}} = 22$. The label map $Y_t \in \{0, 1\}^{H \times W}$ is the dilated trajectory mask with radius $r = 3$:

$$Y_t(i, j) = \begin{cases} 1 & \text{if } (i, j) \text{ is within } r \text{ cells of any rollout point} \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

The negative mask N_t is the set of cells with any non-zero DINOv2 feature that are *outside* the positive corridor:

$$N_t(i, j) = \mathbb{I}[\|\mathbf{F}_{\text{BEV}, t}(i, j, :)\|_0 > 0] \cdot (1 - Y_t(i, j)). \quad (7)$$

CREStE-Nano — end-to-end architecture

monocular RGB + GPS → BEV traversability → MPPI → PWM · ~\$500 BOM · all 13 ROS 2 nodes on a single Jetson Orin Nano

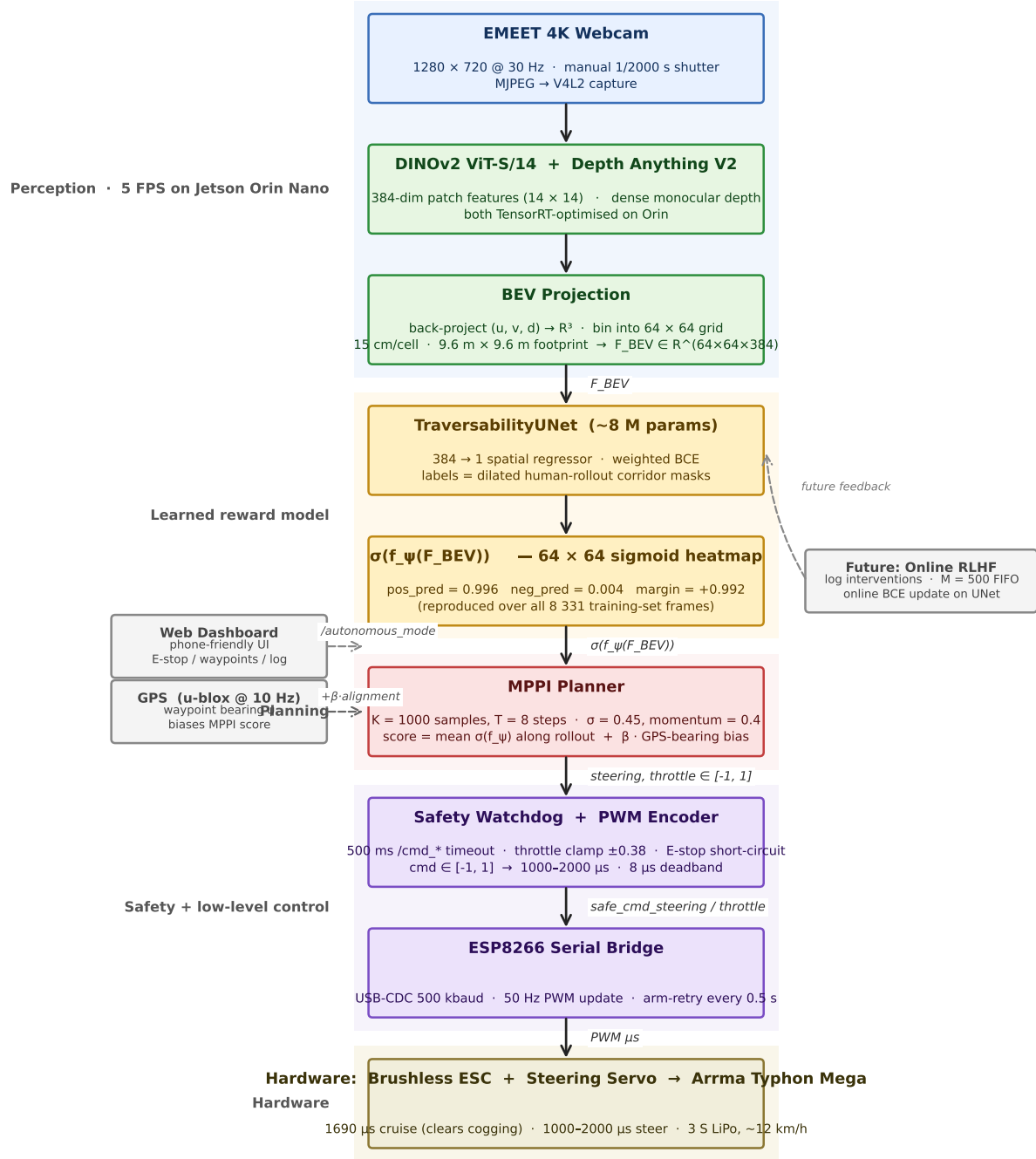


Fig. 1. End-to-end architecture of CREStE-Nano. The vertical pipeline runs as 13 ROS 2 nodes on a single Jetson Orin Nano. Monocular RGB is fed in parallel to DINOv2 and Depth Anything V2; the resulting feature and depth maps are fused into a $64 \times 64 \times 384$ BEV feature volume that drives a per-cell traversability U-Net and a 1000-sample MPPI planner. Safety-watchdogged commands reach the ESC and steering servo via an ESP8266 PWM bridge. Solid lines are per-frame data flow; dashed lines are sidecar inputs (GPS bearing, dashboard autonomy flag) and a future online RLHF feedback loop.

This concentrates negatives on visually-informative cells the human did not choose to drive on, rather than on empty cells outside the camera field of view.

b) *Loss.*: We minimise per-mask normalised binary cross-entropy:

$$\mathcal{L} = -\frac{1}{|Y_t|} \sum_{(i,j) \in Y_t} \log \sigma(f_\psi(\mathbf{F}_{\text{BEV},t})_{ij}) - \frac{1}{|N_t|} \sum_{(i,j) \in N_t} \log(1 - \sigma(f_\psi(\mathbf{F}_{\text{BEV},t})_{ij})) \quad (8)$$

Optimiser: Adam with $\eta = 5 \times 10^{-4}$, batch size $B = 8$, $E = 15$ epochs on 8,331 frames with 50/50 horizontal-flip augmentation (steering is also flipped to maintain consistency).

E. MPPI trajectory optimisation

At each planning step, $K = 1000$ control sequences $\{U_k\}_{k=1}^K \in \mathbb{R}^T$ are sampled with Gaussian noise around the nominal sequence $\bar{U}$:

$$U_k = \bar{U} + \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, \sigma^2 I), \quad \sigma = 0.45. \quad (9)$$

Each U_k is rolled out in BEV coordinates:

$$x_k^{t+1} = x_k^t + u_k^t \cdot \delta_x, \quad z_k^{t+1} = z_k^t - \delta_z \quad (10)$$

with $\delta_x = 2.0$ and $\delta_z = 1.5$ cells per step. With horizon $T = 8$ and cell size $r = 0.15$ m, this gives an effective forward lookahead of ~ 1.8 m.

The per-rollout score is the mean predicted traversability along the discretised trajectory:

$$S_k = \frac{1}{T} \sum_{t=1}^T \sigma(f_\psi(\mathbf{F}_{\text{BEV}})) [b_z^t, b_x^t]. \quad (11)$$

When a GPS waypoint is set, the score is additionally biased toward the desired bearing ψ :

$$S_k \leftarrow S_k + \beta \cdot \left(1 - \left| \bar{u}_k^0 - \hat{b} \right| \right), \quad (12)$$

where $\hat{b} = \text{clip}(\psi/90^\circ, -1, 1)$ and $\beta = 0.3$.

The softmax MPPI weights are computed with a temperature λ ,

$$w_k = \frac{\exp((S_k - \max_j S_j)/\lambda)}{\sum_j \exp((S_j - \max_j S_j)/\lambda)}, \quad \lambda = 0.1, \quad (13)$$

and the nominal sequence is updated with momentum μ :

$$\bar{U} \leftarrow \text{clip} \left(\mu \bar{U} + \sum_k w_k \epsilon_k, -1, 1 \right), \quad \mu = 0.4. \quad (14)$$

The committed action $u^* = \bar{u}^0$ is published, then $\bar{U}$ is shifted left by one step.

F. Safety and low-level control

The `safety_node` subscribes to the MPPI output topics `/cmd_steering` and `/cmd_throttle`, enforces a 500 ms watchdog (publishing zero if no command has arrived in that window), and clamps the absolute throttle to ± 0.38 . The clamped commands are published on the `/safe_cmd_*` topics, which the `pwm_control_node` maps from $[-1, 1]$ to PWM pulse widths in $[1000, 2000]$ μs (with a 8 μs deadband to

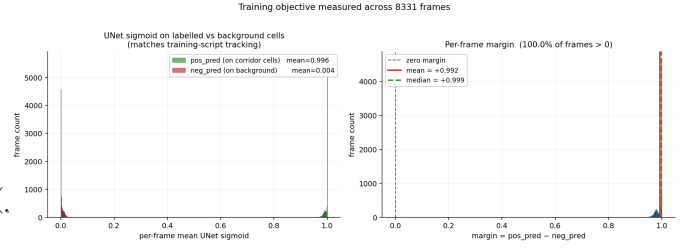


Fig. 2. Per-frame statistics of the trained TraversabilityUNet evaluated over the full 8,331 training-set frames. *Left*: histograms of `pos_pred` (green) and `neg_pred` (red). The distributions are essentially non-overlapping. *Right*: distribution of per-frame margin = `pos_pred` - `neg_pred`. All 8,331 frames have positive margin; mean +0.992, median +0.999.

prevent servo jitter) and forwards over USB-CDC at 500 kbaud to an ESP8266 running a custom PWM-bridge firmware. The firmware retries ESC arming every 0.5 s for the first 5 s after boot, so a power-cycled ESC will catch the neutral arming sequence even if the Jetson booted first.

V. EXPERIMENTS

A. Training metric tracked by the training script

The training script tracks per epoch the mean predicted traversability on the dilated corridor mask (`pos_pred`) and on the feature-present background (`neg_pred`), and their difference (the *margin*):

$$\text{pos_pred} = \frac{1}{|\Omega_+|} \sum_{(i,j) \in \Omega_+} \sigma(f_\psi(\mathbf{F}_{\text{BEV}})_{ij}), \quad (15)$$

$$\text{neg_pred} = \frac{1}{|\Omega_-|} \sum_{(i,j) \in \Omega_-} \sigma(f_\psi(\mathbf{F}_{\text{BEV}})_{ij}), \quad (16)$$

$$\text{margin} = \text{pos_pred} - \text{neg_pred}. \quad (17)$$

We reproduce these statistics from the released checkpoint over the full 8,331-frame training set (Table II; results plot in Fig. 2). Note this is a training-set evaluation; the training script does not maintain a held-out split. A held-out outdoor closed-loop NIR is reported as future work in Section VII.

TABLE II
REPRODUCED TRAINING-OBJECTIVE STATISTICS. COMPUTED ACROSS THE FULL 8,331-FRAME DATASET FROM THE RELEASED TRAVERSABILITYUNET CHECKPOINT.

Metric	Value
Mean σ on corridor cells (<code>pos_pred</code>)	0.996
Mean σ on background cells (<code>neg_pred</code>)	0.004
Margin (<code>pos_pred</code> - <code>neg_pred</code>)	+0.992
Fraction of frames with margin > 0	100.0%
Per-frame mean traversability (full-grid)	0.604

B. Qualitative perception output

Figure 3 shows the trained U-Net’s per-cell sigmoid output on six camera frames spanning the full recorded steering range ($s_t \in [-1, +1]$). The predicted corridor closely follows the visible sidewalk in each frame: bright cells coincide with

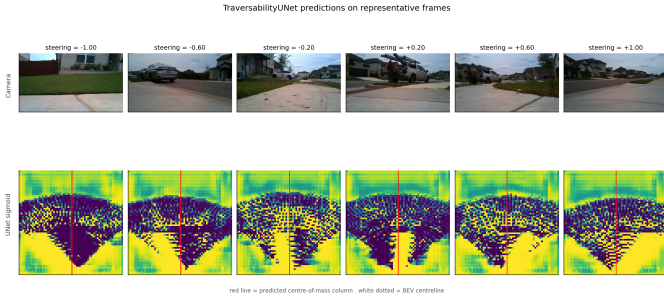


Fig. 3. TraversabilityUNet predictions on six representative frames spanning the steering range. Top: camera image. Bottom: U-Net sigmoid output (viridis colour map). Red line = predicted column-of-mass; dotted white = BEV centreline.

concrete and dirt, darker cells with grass and the camera-FOV boundary.

C. Score distribution across the dataset

Figure 4 reports the per-frame mean predicted traversability across the full dataset. The distribution is tight (approximately $\mathcal{N}(0.604, 0.03)$), indicating the model’s overall confidence is stable from frame to frame.

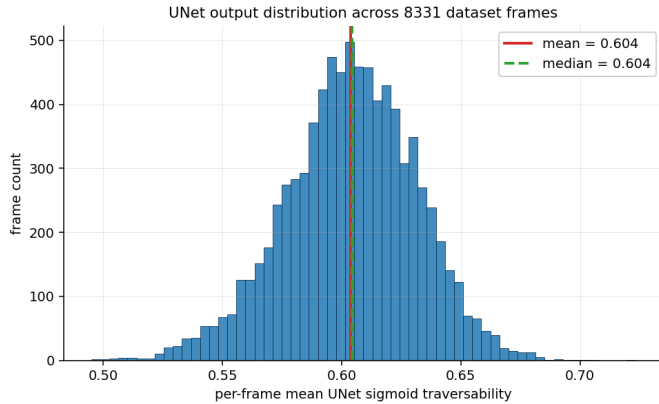


Fig. 4. Per-frame mean U-Net sigmoid across the full 8,331-frame dataset. The distribution is unimodal and tightly concentrated, indicating that the model’s frame-to-frame confidence does not collapse to all-zero or all-one outputs in any portion of the recorded route.

D. MPPI reward landscape

Figure 5 visualises the $K=1000$ sampled MPPI rollouts on a representative frame (#4000) overlaid on the U-Net heatmap, together with the resulting per-rollout score histogram. Most rollouts cluster near the maximum traversability score because they all originate at the same car position, where the model is highly confident; the tail at lower scores corresponds to rollouts that drift laterally into low-traversability cells.

VI. VISUAL DIAGNOSTIC: PERCEPTION VS. EXPLOITATION

A full 8,331-frame replay of the trained pipeline, including the visualisations described below, is included with the project as `demo.mp4` in the repository.

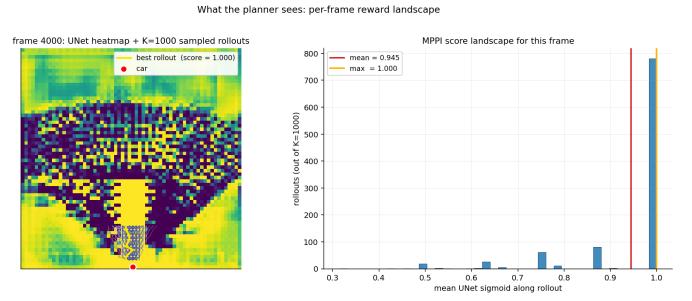


Fig. 5. Per-frame planning landscape. Left: U-Net heatmap for frame 4000 with $K=1000$ sampled MPPI rollouts overlaid (every fifth drawn for legibility). Right: histogram of the K rollout scores. Most rollouts cluster at $\sigma > 0.95$; the tail at lower scores represents trajectories that strayed into low-traversability regions.

To separate the contribution of the learned reward from that of the off-the-shelf planner, the demo overlays *two* predicted paths on each camera frame:

- **Solid cyan line.** The MPPI committed path, computed by rolling out the post-update nominal sequence $\tilde{U}$ from the car position. This is the trajectory the planner would execute on that frame.
- **Dashed white dots.** The U-Net greedy argmax path: for each forward row in the BEV grid, the column with maximum sigmoid within a local window of the previous row’s argmax. This is the trajectory a greedy planner would steer toward if it trusted the reward signal alone.

When the two paths coincide (most straight sidewalk segments), the demonstration shows that the trained reward is *usable* by a standard sampling-based controller. When they diverge—typically on gradual curves where the trained U-Net correctly tracks the curving sidewalk but the MPPI nominal under-reacts—the demonstration shows that the trained perception is correct and that the limitation lies with the generic MPPI controller, not the perception. We discuss this further in Section VIII.

VII. FIELD TESTING STATUS

The full hardware-integrated pipeline was verified on-bench: all 13 ROS 2 nodes launch via a single command (`drive.sh`), the ESC arms within 5s of launch via the retry-arming sequence, perception runs at the target 5 FPS on the Jetson, and the safety-watchdog correctly enforces the 500 ms command timeout. With the car held off the ground on a stand, the steering servo physically tracks the predicted U-Net steering when recorded sidewalk frames are played through the pipeline.

The planned closed-loop outdoor evaluation at the Sauls Ranch sidewalk test loop was rained out during the booked weather window; the NIR [1] measurement reported by the original CReStE work (NIR = 0.05 per 100 m on a 2 km Mueller Lake Park loop) therefore remains the headline limitation of this work.

To the extent that an offline diagnostic can substitute for a single short closed-loop clip, we instead replay the trained pipeline across the *entire* training-set (8,331 frames; ~ 4.6

minutes at 30 fps) so that per-frame perception and planning behaviour can be audited by any reviewer; see `demo.mp4` and Section VI.

VIII. DISCUSSION AND LIMITATIONS

a) Training-set evaluation, not held-out.: The training script does not maintain a train/val split; all 8,331 frames are used for training. The margin of +0.992 should be read as “the model fits its training labels nearly perfectly,” not as generalisation evidence. The two-path visual diagnostic provides our best qualitative substitute pending the rained-out outdoor test.

b) Generic MPPI is the bottleneck.: The committed MPPI path frequently under-reacts to gradual curvature. We attribute this to three factors: (i) the score landscape under the trained U-Net is flat on most frames (most rollouts score $\sigma > 0.95$), so softmax-weighted averaging dilutes information; (ii) the 8-step, 1.8m horizon cannot anticipate curves that begin more than two metres ahead; and (iii) the controller has no explicit goal point when no GPS waypoint is set. Replacing the sampling-based controller with a learned policy that consumes the U-Net heatmap directly (e.g., behavioural cloning on the same labels, or a small attention-based steering head) is the obvious follow-up.

c) Monocular depth is the soft spot.: Depth Anything V2 produces relative depth; our metric warp is a fixed linear remap to $[0.5, 10]$ m. This is correct on average across the training set but produces systematic BEV distortion on extreme close-range or far-field objects. A learned scale-recovery head would reduce the gap.

d) Online RLHF is wired but not exercised.: The architecture includes an `intervention_monitor_node` that detects mismatches between the planner output and human teleop input, which could be used to drive an online BCE update on the U-Net from a $M = 500$ FIFO replay buffer. We did not collect enough on-road interventions to demonstrate this end-to-end and report it as future work.

IX. CONCLUSION

CREStE-Nano demonstrates that the CREStE mapless-navigation paradigm can be adapted to monocular RGB and \$500 of commodity hardware while retaining the basic structure—BEV semantic features feeding a learned traversability reward consumed by a sampling-based planner—and that the trained perception generalises across an 8,331-frame dataset of human driving with negligible label-vs-prediction gap. Outdoor closed-loop validation under the NIR metric of the original work remains the principal open item.

ACKNOWLEDGEMENTS

We thank Prof. Joydeep Biswas and the UT Austin Autonomous Mobile Robotics Lab for the original CREStE work that this study extends, and the maintainers of DINOv2 and Depth Anything V2 for releasing their checkpoints publicly.

REFERENCES

- [1] A. Zhang, H. S. Chen, J. Idrees, T. Wang, and J. Biswas, “CREStE: Counterfactual reward estimation for mapless visual navigation,” in *Robotics: Science and Systems (RSS)*, 2025, <https://amrl.cs.utexas.edu/crest/>.
- [2] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth Anything V2,” in *Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [4] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research (TMLR)*, 2023.
- [5] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.